

GlobalMart OS

Ecosistema de Microservicios para Minimarkets

POS | ERP | Gestión Multi-Tenant

Informe de Perfil de Proyecto

Taller de Integración III - INFO1166

Equipo Diego Leiva · Sebastián Pereda · Martín Sanhueza ·
Rodrigo Pedraza · Daniel Prado · Nicolás Martínez

Fecha 9 de septiembre de 2026

Índice

1. Justificación del Proyecto	3
1.1. Situación Sin Solución	3
1.2. Situación Con Solución	4
2. Objetivos Principales	6
2.1. Objetivo General	6
2.2. Objetivos Específicos	6
3. Arquitectura y Diseño del Sistema	7
3.1. Vista General de Componentes	7
3.2. Actores del Sistema	7
3.3. Ecosistema de Microservicios	8
3.4. Arquitectura de Eventos (Kafka)	9
3.5. Diseño Multi-Tenant y Seguridad	9
3.6. Integraciones Externas y Hardware	9
4. Reglas de Negocio	11
4.1. Must — Obligatorias	11
4.2. Should — Importantes	11
4.3. Could — Deseables	12
4.4. Won't — Fuera de Alcance (v1.0)	12
5. Análisis de Requerimientos	13
5.1. Requerimientos Funcionales	13
5.2. Requerimientos No Funcionales	15
6. Casos de Uso	16
6.1. Must	16
6.2. Should	16
6.3. Could	17
6.4. Won't	17
7. Matriz de Trazabilidad	18
8. Backlog del Producto	20
8.1. Épicas	20
8.2. Historias de Usuario — Sprint 1	20
9. Alcance Preliminar: Plan de Sprints (MVP en 3 Iteraciones)	21
10. Roadmap — Sprints Futuros	26
11. Restricciones y Recursos	26
11.1. Equipo de Desarrollo	26
11.2. Restricción de Capacidad	27
11.3. Stack Tecnológico Oficial	27

11.4. Recursos de Hardware	27
12.Alineación con el Estándar SWEBOK	28
13.Estructura del Repositorio para Documentación	28
14.Riesgos Identificados	30

1. Justificación del Proyecto

El mercado minorista de pequeño y mediano formato (minimarkets, almacenes de barrio y cadenas regionales) enfrenta una brecha tecnológica crítica: las soluciones comerciales existentes son o bien demasiado costosas para el segmento PYME, o bien sistemas monolíticos heredados que no escalan ni permiten la gestión de múltiples sucursales de forma integrada.

GlobalMart OS nace como respuesta directa a esta problemática. El sistema provee un ecosistema completo de microservicios diseñado para cubrir el ciclo de vida operativo completo de un minimarket moderno:

- **Punto de Venta (POS):** procesamiento de ventas en tiempo real, soporte de hardware de pesaje para productos a granel y emisión de tickets.
- **ERP Integrado:** gestión de catálogo de productos, precios por sucursal, cadena de suministros e importaciones con cálculo de *Landed Cost*.
- **Gestión Multi-Tenant:** arquitectura que permite a múltiples minimarkets operar de forma aislada sobre la misma plataforma, con configuración regional propia (moneda, zona horaria, formato de fecha, idioma).
- **Analítica en tiempo real:** tableros de indicadores clave (KPI), alertas proactivas de bajo stock y vencimiento de productos.

Desde la perspectiva técnica, el proyecto constituye una oportunidad formativa de alto valor para el equipo: se aplican patrones de arquitectura empresarial modernos (*Domain-Driven Design*, *Database-per-Service*, *Event-Driven Architecture*) sobre un stack tecnológico de demanda creciente en la industria (**.NET 8**, **Apache Kafka**, **Kong**, **Electron + React**).

1.1. Situación Sin Solución

Los minimarkets, especialmente en Latinoamérica y mercados emergentes, operan en un entorno de alta competencia con márgenes ajustados. Sin embargo, la gran mayoría gestiona sus operaciones de forma **fragmentada, manual o con herramientas no integradas**, lo que genera ineficiencias críticas en cada área del negocio.

Área	Problema Actual
Punto de Venta	Cajas registradoras sin integración fiscal. Cobros manuales propensos a errores. Sin pasarela de pago integrada.
Inventario	Control en papel o Excel. Sin alertas de stock mínimo. Productos caducados se venden por falta de control FEFO.
Cumplimiento Fiscal	Boletas y facturas emitidas manualmente. Alto riesgo de errores en declaraciones tributarias. Sin adaptación por país.
Multi-Sucursal	Cada sucursal opera como una isla aislada. Sin visibilidad centralizada del negocio.
Compras e Importaciones	Órdenes de compra en correo o papel. Sin cálculo de costo real (flete, aranceles, seguros).
Fidelización	Sin programa de lealtad. Sin conocimiento del cliente. Pérdida de clientes recurrentes.
Reportes	Sin datos en tiempo real. Decisiones basadas en información desactualizada o incompleta.
Escalabilidad	Cuando el negocio crece a nuevas sucursales o países, el sistema actual no escala.

Consecuencias:

- **Pérdidas económicas** por mermas no registradas y stock caducado no detectado.
- **Multas fiscales** por documentos tributarios incorrectos o emitidos fuera de plazo.
- **Pérdida de ventas** por desabastecimiento no anticipado.
- **Incapacidad de escalar** a nuevos mercados o países sin reemplazar todo el sistema.
- **Experiencia de cliente deficiente** por colas largas, errores de cobro y sin beneficios de fidelidad.

1.2. Situación Con Solución

GlobalMart OS es un sistema operativo de gestión integral (**POS + ERP**) diseñado específicamente para minimarkets, con arquitectura **Multi-Tenant** que permite a cada negocio operar de forma aislada e independiente dentro de la misma plataforma.

Área	Solución GlobalMart OS
Punto de Venta	App de escritorio nativa (Electron) con escaneo de productos, integración con balanza física, terminal POS y pasarela de pago.
Inventario	Control de stock en tiempo real con descuento automático por venta (vía Kafka). Regla FEFO para perecederos. Alertas automáticas de stock mínimo y caducidad.
Cumplimiento Fiscal	Integración directa con entidades fiscales (SII, AFIP, IRS). Cálculo automático de impuestos compuestos. Emisión de DTE con folio electrónico validado.
Multi-Sucursal	Arquitectura Multi-Tenant con gestión centralizada de sucursales. Dashboard unificado con filtros por sucursal.
Compras e Importaciones	Módulo de Supply Chain con cálculo de Costo Landed (flete + aranceles + seguros). Tracking de envíos con sistema de transporte 3PL.
Fidelización	Programa de lealtad completo: acumulación de puntos, tiers de membresía, cupones y canje de productos.
Reportes	Dashboards en tiempo real alimentados por eventos Kafka. Exportación a PDF y Excel. KPIs siempre actualizados.
Escalabilidad	8 microservicios independientes. Configuración de localización por tenant (país, moneda, idioma, reglas fiscales).

Valor Generado:

- **Reducción de mermas** por control FEFO y alertas de caducidad.
- **Cero multas fiscales** por cumplimiento automático y validado.
- **Aumento de ventas** por mejor experiencia en caja y programa de lealtad.
- **Expansión sin fricción** a nuevas sucursales o países con configuración de tenant.
- **Decisiones basadas en datos** con dashboards y KPIs en tiempo real.

2. Objetivos Principales

2.1. Objetivo General

Diseñar, implementar y validar un sistema de gestión integral para minimarkets basado en una arquitectura de microservicios, que permite la operación simultánea de múltiples sucursales bajo un modelo multi-tenant, cumpliendo con los estándares de calidad, seguridad y trazabilidad requeridos por el contexto académico de Taller de Integración III.

2.2. Objetivos Específicos

1. **Infraestructura y Seguridad:** Implementar una capa de API Gateway con *Kong 3.6 DB-less* que centralice el enrutamiento, la validación de tokens JWT y la limitación de tasa (*rate-limiting*), garantizando que ningún microservicio sea accesible directamente desde el cliente.
2. **Identidad Multi-Tenant:** Desarrollar el servicio *Tenant & Identity* en ASP.NET Core 8 con autenticación JWT (`access_token` de 8 horas, `refresh_token` de 7 días) y autorización basada en roles (SUPER_ADMIN, ADMINISTRADOR, CAJERO, REPONEDOR).
3. **Gestión de Catálogo y Precios:** Construir un servicio de catálogo que soporte jerarquía de categorías, unidades de medida, variaciones de precio por sucursal y productos de peso variable.
4. **Punto de Venta Desktop:** Desarrollar una aplicación de escritorio multiplataforma con *Electron + React 18*, capaz de operar el flujo completo de venta: apertura de turno, escaneo de productos, integración con balanza, procesamiento de pago y cierre de turno.
5. **Inventario con Regla FEFO:** Implementar el servicio de inventario aplicando la regla *First Expired, First Out* (FEFO) para el descuento de stock, con trazabilidad completa por lote y alertas automatizadas.
6. **Arquitectura de Eventos:** Integrar *Apache Kafka* como bus de eventos asíncronos, implementando al menos los eventos críticos: `sale.completed`, `stock.low` y `product.expiring_soon`.
7. **Calidad de Software:** Alcanzar una cobertura mínima del 70% en pruebas unitarias y de integración mediante *xUnit*, y validar los flujos críticos de la aplicación Electron con *Playwright E2E*.

3. Arquitectura y Diseño del Sistema

La solución tecnológica de GlobalMart OS se basa en patrones de arquitectura modernos, combinando *Domain-Driven Design* (DDD), *Database-per-Service* y *Event-Driven Architecture* para proveer robustez y escalabilidad global.

3.1. Vista General de Componentes

El sistema se compone de múltiples capas interconectadas:

- **Frontend (Desktop):** Aplicación de escritorio multiplataforma (*Desktop-First*) desarrollada con *Electron*, *React 18* y *TypeScript 5+*.
- **API Gateway:** *Kong 3.6 (DB-less)* actúa como punto de entrada único, gestionando el enrutamiento, validación JWT y rate-limiting.
- **Backend (Microservicios):** 8 microservicios RESTful desarrollados en *ASP.NET Core 8*.
- **Bases de Datos:** Cada microservicio posee su propia instancia aislada de *PostgreSQL 16*.
- **Bus de Eventos:** *Apache Kafka* maneja la comunicación asíncrona entre microservicios para operaciones críticas (ej. descuento de stock).

3.2. Actores del Sistema

El ecosistema interactúa con un total de 13 actores, divididos en actores humanos (usuarios del sistema con herencia de roles) y actores de sistema o hardware.

Actores Humanos

Actor	Tipo	Descripción y Capacidades
Cajero	Primario	Opera el POS. Abre/cierra turno, gestiona carrito y cobros.
Reponedor	Primario	Gestiona inventario. Aplica FEFO, mermas y recepciones.
Administrador	Primario	Configura el tenant, usuarios, reportes y órdenes de compra.
Super Admin	Secundario	Dueño global. Gestiona tenants y mantenimiento macro.
Cliente Afiliado	Secundario	Acumula/canjea puntos en POS e interactúa con el programa de lealtad.

Actores de Sistema y Hardware

Actor	Tipo	Protocolo / Integración
Balanza Física	Hardware	Serial/USB (COM). Envía peso de productos a granel.
Terminal POS	Hardware	USB/LAN. Agrupa lector tarjetas, impresora, cajón y display.
Entidad Fiscal	Ext. (API)	SII/AFIP/IRS. Valida folios y DTE.
Pasarela Pago	Ext. (API)	Stripe/Transbank. Procesa tarjetas de crédito/débito.
Sistema 3PL	Ext. (API)	Courier de transporte externo. Cotizaciones y tracking.
Mensajería	Ext. (API)	Twilio, SendGrid, Firebase para envío de SMS/Email/Push.
Servicio FX	Ext. (API)	Fixer.io. Extrae tasas de cambio de moneda en tiempo real.
Kafka	Interno	Bus de mensajería asíncrona (Apache Kafka Protocol).

3.3. Ecosistema de Microservicios

Para garantizar alta cohesión y bajo acoplamiento, el backend se divide en los siguientes 8 microservicios independientes:

1. **MS-1 Tenant & Identity Service:** Gestión de autenticación (JWT), usuarios, roles RBAC y configuración general de cada minimarket (localización, moneda, huso horario).
2. **MS-2 Tax & Compliance Service:** Motor de cálculo de impuestos compuestos y comunicación directa con entidades fiscales para emisión de Boletas/Facturas electrónicas validadas.
3. **MS-3 Catalog & Pricing Service:** Gestión del maestro de productos, conversiones de unidades de medida (UOM), códigos de barras y reglas de precios dinámicos por sucursal.
4. **MS-4 Warehouse & Inventory Service:** Control de stock. Implementa la regla FEFO obligatoria para perecederos y emite alertas automáticas de caducidad y stock mínimo.
5. **MS-5 POS & Cart Service:** Núcleo operativo en mostrador. Orquesta el turno de caja, el carrito de compras y las interacciones con el hardware (balanza, pasarela de pago).
6. **MS-6 Supply Chain & Import Service:** Ciclo de compras, cálculo avanzado de Costo Landed (flete, aduanas, seguros convertidos a moneda local) y tracking logístico con 3PL.
7. **MS-7 Analytics & Notification Service:** Consolida eventos de Kafka para alimentar dashboards en tiempo real y disparar notificaciones multicanal (Push/SMS/Email).
8. **MS-8 Loyalty & Customer Service:** Gestión integral del cliente, acumulación de puntos por compras, tiers de membresía y canje de cupones promocionales.

3.4. Arquitectura de Eventos (Kafka)

El sistema utiliza Apache Kafka para desacoplar transacciones críticas. La comunicación asíncrona garantiza que el POS no se bloquee esperando operaciones secundarias. El flujo principal se detalla a continuación:

- **sale.completed:** Producido por *MS-5 POS* al finalizar un cobro exitoso.
 - Consumido por *MS-4* para descontar stock asincrónamente.
 - Consumido por *MS-2* para el registro contable tributario.
 - Consumido por *MS-8* para sumar puntos de lealtad al cliente (si está identificado).
 - Consumido por *MS-7* para actualizar métricas financieras y KPIs.
- **stock.alert / expiry.alert:** Producido por *MS-4* cuando se rompen umbrales de stock mínimo o un lote está próximo a vencer. Consumido por *MS-7* para disparar alertas a los encargados.
- **fx.rate.updated:** Producido por *MS-1* al detectar variaciones de moneda externa. Consumido por *MS-3* (catálogo) y *MS-6* (compras) para reajustar precios y costos logísticos dinámicamente.

3.5. Diseño Multi-Tenant y Seguridad

- **Aislamiento de Datos:** El sistema implementa un patrón Multi-Tenant estricto a nivel lógico. Cada tabla en cada base de datos PostgreSQL contiene una columna `tenant_id`. Un *Middleware* en ASP.NET Core intercepta cada petición HTTP, extrae el `tenant_id` del JWT validado, y lo inyecta automáticamente en los queries de *Entity Framework Core*, garantizando que ningún query exponga datos cruzados.
- **Gestión Multi-Rol (RBAC):** Los usuarios pueden poseer múltiples roles simultáneos. El JWT emitido incluye la lista de roles del usuario y un campo `active_role`. En caso de requerir privilegios puntuales (ej. un Cajero necesita anular una venta), el frontend (Electron) permite una "escalada de rol temporal" solicitando contraseña al Administrador de turno, registrando fielmente este evento en el log de auditoría.

3.6. Integraciones Externas y Hardware

- **Hardware de Mostrador (Electron):** La lectura de la balanza de peso variable se realiza procesando el puerto serial (RS-232/USB HID) a través del proceso *main* de Electron usando el paquete `serialport`, exponiendo el peso dinámico al *renderer* vía `contextBridge` a través de IPC.
- **Pasarelas de Pagos:** Para asegurar cumplimiento del estándar *PCI DSS*, el sistema jamás procesa los números de tarjeta localmente. *MS-5* se comunica mediante API HTTPS con servicios como Stripe o Transbank, delegando la captura segura del chip o NFC al Terminal POS físico.
- **Cumplimiento Fiscal Integrado:** *MS-2* encapsula la complejidad legal de cada país, implementando SOAP para la AFIP (Argentina) o REST con firma electrónica para el

SII (Chile). Se manejan colas de reintentos con *backoff* exponencial automático si el servicio fiscal gubernamental sufre caídas de sistema.

4. Reglas de Negocio

Las siguientes reglas de negocio rigen el comportamiento del sistema, priorizadas bajo el esquema MoSCoW (*Must, Should, Could, Won't*).

4.1. Must — Obligatorias

ID	Regla de Negocio
RN-01	Cada tenant (minimarket) debe estar completamente aislado: datos, configuración y usuarios son independientes entre tenants.
RN-02	Todo pago con tarjeta (débito/crédito) debe pasar por la pasarela de pago externa. Nunca se procesa localmente.
RN-03	Toda venta completada debe generar un documento tributario válido (boleta o factura) según la normativa fiscal del país del tenant.
RN-04	El stock de un producto debe descontarse automáticamente al completar una venta exitosa.
RN-05	Los productos perecederos deben gestionarse bajo la regla FEFO (First Expired, First Out): el lote con menor fecha de vencimiento se despacha primero.
RN-06	Solo se pueden procesar ventas si existe un turno de caja abierto por el cajero en sesión.
RN-07	Todo usuario debe estar autenticado mediante JWT para realizar cualquier operación en el sistema.
RN-08	El token JWT debe incluir el tenant activo y los roles del usuario para cada sesión.

4.2. Should — Importantes

ID	Regla de Negocio
RN-09	El Costo Landed de una importación debe calcularse como: Costo Producto + Flete + Aranceles + Seguros + Gastos Aduaneros, convertido a moneda local.
RN-10	Las alertas de stock mínimo deben emitirse cuando el stock disponible caiga por debajo del umbral configurado, antes de llegar a cero.
RN-11	Los precios de un producto pueden variar por sucursal dentro del mismo tenant.
RN-12	Un usuario puede tener múltiples roles activos (ej: Cajero + Reponedor). El sistema registra el rol activo al momento de cada acción en el log de auditoría.
RN-13	La separación de funciones puede configurarse por tenant: el creador de una Orden de Compra no puede ser el mismo que la aprueba.
RN-14	Las devoluciones parciales o totales requieren permiso explícito del rol Administrador o permiso especial asignado al cajero.

4.3. Could — Deseables

ID	Regla de Negocio
RN-15	Los puntos de lealtad expiran automáticamente si el cliente no registra actividad en un período mayor a 12 meses.
RN-16	El tipo de cambio de cada moneda debe actualizarse automáticamente desde un servicio externo (Fixer.io) cuando la variación supera el umbral configurado por el tenant.
RN-17	Los multiplicadores de puntos de lealtad pueden configurarse por categoría de producto (ej: lácteos x2 puntos).
RN-18	Los descuentos manuales en carrito requieren que el cajero tenga el permiso <code>discount.apply</code> asignado explícitamente en su rol.

4.4. Won't — Fuera de Alcance (v1.0)

ID	Regla de Negocio
RN-19	No se gestionará inventario en consignación en esta versión.
RN-20	No se soportará multi-moneda dentro de una misma transacción de venta.
RN-21	No habrá integración con marketplaces online (MercadoLibre, Amazon, etc.).
RN-22	No se incluirá gestión de nómina ni recursos humanos.

5. Análisis de Requerimientos

5.1. Requerimientos Funcionales

Must

ID	Requerimiento
RF-01	El sistema debe permitir el inicio de sesión con usuario y contraseña, generando un JWT con tenant, roles y tiempo de expiración.
RF-02	El sistema debe permitir la apertura de turno de caja registrando el monto de fondo inicial.
RF-03	El sistema debe permitir el cierre de turno con declaración de billetes y monedas para cuadre.
RF-04	El sistema debe permitir escanear o buscar productos para agregarlos al carrito de compras.
RF-05	El sistema debe calcular el total de la venta con impuestos según las reglas fiscales del tenant.
RF-06	El sistema debe procesar cobros en efectivo con cálculo automático de vuelto.
RF-07	El sistema debe procesar cobros con tarjeta enviando la solicitud a la pasarela de pago y procesando la respuesta (aprobación o rechazo).
RF-08	El sistema debe emitir el documento tributario (boleta/factura electrónica) al completar una venta.
RF-09	El sistema debe descontar el stock de cada producto vendido de forma asíncrona al completar la venta.
RF-10	El sistema debe permitir registrar la recepción de mercancía aplicando automáticamente la regla FEFO.
RF-11	El sistema debe permitir crear, editar y eliminar productos en el catálogo con código de barras/QR, precio y UOM.
RF-12	El sistema debe permitir configurar el tenant definiendo país, moneda, idioma y zona horaria.
RF-13	El sistema debe permitir gestionar usuarios con creación, edición, desactivación y asignación de roles (RBAC).
RF-14	El sistema debe implementar aislamiento multi-tenant asegurando que ningún tenant acceda a datos de otro.

Should

ID	Requerimiento
RF-15	El sistema debe integrarse con balanza física (USB/Serial) para capturar el peso de productos a granel directamente en el carrito.
RF-16	El sistema debe generar alertas automáticas de stock mínimo y próxima caducidad, publicándolas como eventos Kafka hacia el servicio de notificaciones.
RF-17	El sistema debe ofrecer dashboards en tiempo real de ventas, inventario y KPIs financieros.
RF-18	El sistema debe permitir crear órdenes de compra con selección de proveedor, ítems y cálculo de Costo Landed.
RF-19	El sistema debe permitir gestionar múltiples sucursales dentro del mismo tenant con inventarios independientes.
RF-20	El sistema debe soportar usuarios con múltiples roles simultáneos y cambio de rol activo en sesión.
RF-21	El sistema debe permitir configurar precios por sucursal y definir promociones con rango de fechas.
RF-22	El sistema debe permitir anular ventas y procesar devoluciones (total o parcial) con reembolso a pasarela si corresponde.

Could

ID	Requerimiento
RF-23	El sistema debe implementar un programa de lealtad con acumulación de puntos por compra, tiers de membresía y canje por descuentos o productos.
RF-24	El sistema debe integrarse con un Servicio de Tipos de Cambio (Fixer.io) para actualizar automáticamente las tasas FX del tenant.
RF-25	El sistema debe enviar notificaciones por SMS, email y push vía proveedor externo (Twilio/SendGrid) para alertas y estados de cuenta de lealtad.
RF-26	El sistema debe permitir exportar reportes de ventas, mermas y productos a PDF y Excel.
RF-27	El sistema debe permitir rastrear importaciones en tiempo real mediante integración con sistema de transporte 3PL.
RF-28	El sistema debe permitir emitir y validar cupones de descuento en el punto de venta.

Won't

ID	Requerimiento
RF-29	No se desarrollará una aplicación móvil para clientes afiliados.
RF-30	No se implementará integración con e-commerce o marketplaces.
RF-31	No se incluirá gestión de nómina ni control de asistencia.
RF-32	No se soportará inventario en consignación en v1.0.

5.2. Requerimientos No Funcionales

Must

ID	Requerimiento
RNF-01	Aislamiento de datos: ningún query puede retornar datos de otro tenant. Implementado con discriminador <code>tenant_id</code> en cada tabla.
RNF-02	Rendimiento del POS: el tiempo de respuesta desde escaneo de producto hasta visualización en carrito debe ser < 2 segundos.
RNF-03	Disponibilidad: el sistema debe operar con una disponibilidad $\geq 99.5\%$ (downtime máx. ~ 3.6 hrs/mes).
RNF-04	Seguridad: los datos sensibles (tokens, datos de pago) deben cifrarse en tránsito (TLS 1.3) y en reposo (AES-256).

Should

ID	Requerimiento
RNF-05	Auditoría: todas las operaciones críticas (login, venta, anulación, ajuste de stock) deben registrarse en un log de auditoría inmutable.
RNF-06	Multiplataforma: la aplicación Electron debe ejecutarse en Windows, macOS y Linux sin cambios de código.
RNF-07	Cobertura de tests: el backend debe mantener una cobertura de tests unitarios $\geq 80\%$ en lógica de negocio crítica.

Could

ID	Requerimiento
RNF-08	Internacionalización (i18n): el frontend debe soportar múltiples idiomas configurables por tenant (ES, EN, PT como mínimo).
RNF-09	Modo offline parcial: el POS debe poder procesar ventas en efectivo sin conexión a internet, sincronizando al reconectar.

6. Casos de Uso

Los identificadores de casos de uso (CU) corresponden a los definidos en el diagrama UML del proyecto.

6.1. Must

ID	Caso de Uso	Actor(es)
CU-01	Autenticar Usuario (Login + MFA)	Cajero, Reponedor, Admin
CU-02	Abrir Turno de Caja	Cajero
CU-03	Cerrar Turno y Cuadre de Caja	Cajero, Admin
CU-04	Gestionar Carrito de Compras	Cajero
CU-05	Cobrar en Efectivo	Cajero, Terminal POS
CU-06	Cobrar con Tarjeta vía Pasarela	Cajero, Terminal POS, Pasarela
CU-07	Emitir Documento Tributario	Sistema, Entidad Fiscal
CU-08	Descontar Stock por Venta (async)	Kafka, Sistema
CU-09	Recibir Mercancía con FEFO	Reponedor
CU-10	Gestionar Catálogo de Productos	Admin
CU-11	Configurar Tenant	Admin, Super_Admin
CU-12	Gestionar Usuarios y Roles (RBAC)	Admin

6.2. Should

ID	Caso de Uso	Actor(es)
CU-13	Integrar Balanza Física	Cajero, Balanza Física
CU-14	Anular Venta y Devolución	Cajero, Admin, Pasarela
CU-15	Generar Alertas de Stock y Caducidad	Sistema, Kafka
CU-16	Visualizar Dashboards y KPIs	Admin, Reponedor
CU-17	Crear Orden de Compra con Landed Cost	Admin
CU-18	Gestionar Sucursales	Admin
CU-19	Procesar Pago Mixto	Cajero
CU-20	Configurar Precios y Promociones	Admin
CU-21	Cambiar Rol Activo / Escalada de Rol	Usuario Multi-Rol
CU-22	Consultar y Ajustar Stock	Reponedor, Admin

6.3. Could

ID	Caso de Uso	Actor(es)
CU-23	Gestionar Programa de Lealtad	Admin
CU-24	Identificar Cliente Afiliado en POS	Cajero, Cliente Afiliado
CU-25	Acumular y Canjear Puntos	Cajero, Cliente Afiliado
CU-26	Actualizar Tipo de Cambio (FX)	Servicio FX, Sistema
CU-27	Enviar Notificaciones Multi-Canal	Sistema, Proveedor Mensajería
CU-28	Exportar Reportes PDF / Excel	Admin
CU-29	Rastrear Importación	Admin, Sistema de Transporte
CU-30	Emitir y Validar Cupones	Admin, Cajero

6.4. Won't

ID	Caso de Uso	Razón
CU-31	App móvil para cliente afiliado	Fuera de alcance v1.0
CU-32	Publicar productos en marketplace	Fuera de alcance v1.0
CU-33	Gestión de nómina	Fuera de alcance v1.0

7. Matriz de Trazabilidad

La siguiente matriz relaciona cada Regla de Negocio con sus Requerimientos y Casos de Uso asociados, permitiendo verificar que toda regla del negocio quede cubierta por al menos un requerimiento y un caso de uso implementable.

Regla de Negocio	Requerimiento(s)	Caso(s) de Uso	Prioridad
RN-01 Aislamiento tenant	RF-14, RNF-01	CU-11	Must
RN-02 Pago tarjeta vía pasarela	RF-07	CU-06	Must
RN-03 Documento tributario por venta	RF-08	CU-07	Must
RN-04 Descuento stock automático	RF-09	CU-08	Must
RN-05 Regla FEFO perecederos	RF-10	CU-09	Must
RN-06 Turno abierto para ventas	RF-02	CU-02, CU-04	Must
RN-07 Usuario autenticado con JWT	RF-01	CU-01	Must
RN-08 JWT con tenant y roles	RF-01, RF-13	CU-01, CU-12	Must
RN-09 Cálculo Costo Landed	RF-18	CU-17	Should
RN-10 Alertas stock antes de cero	RF-16	CU-15	Should
RN-11 Precios por sucursal	RF-21	CU-20	Should
RN-12 Múltiples roles / rol activo	RF-20	CU-21, CU-12	Should
RN-13 Separación de funciones OC	RF-18, RF-13	CU-17, CU-12	Should
RN-14 Devolución requiere permiso	RF-22	CU-14	Should
RN-15 Expiración puntos 12 meses	RF-23	CU-25	Could
RN-16 FX automático con umbral	RF-24	CU-26	Could
RN-17 Multiplicadores puntos por cat.	RF-23	CU-23	Could
RN-18 Descuento manual con permiso	RF-21, RF-13	CU-04, CU-12	Could

Regla de Negocio	Requerimiento(s)	Caso(s) de Uso	Prioridad
RN-19 Sin inventario consignación	RF-32	—	Won't
RN-20 Sin multi-moneda en transacción	RF-14	—	Won't

8. Backlog del Producto

8.1. Épicas

ID	Épica	Prioridad
EP-01	Autenticación y Gestión de Identidad	Must
EP-02	Punto de Venta (POS Core)	Must
EP-03	Catálogo de Productos	Must
EP-04	Gestión de Inventario	Must
EP-05	Cumplimiento Fiscal	Must
EP-06	Gestión de Sucursales y Configuración	Should
EP-07	Cadena de Suministro y Compras	Should
EP-08	Analytics y Dashboards	Should
EP-09	Programa de Lealtad	Could
EP-10	Notificaciones y Mensajería	Could
EP-11	Tipos de Cambio FX	Could

8.2. Historias de Usuario — Sprint 1

ID	Historia de Usuario	Puntos	Prioridad
US-01	Como admin , quiero configurar mi tenant (país, moneda, idioma) para que el sistema opere según mi región.	5	Must
US-02	Como admin , quiero crear usuarios y asignarles roles para controlar quién accede al sistema.	5	Must
US-03	Como cajero , quiero iniciar sesión con usuario y contraseña para acceder al POS.	3	Must
US-04	Como cajero , quiero abrir un turno de caja registrando el monto inicial para comenzar a operar.	3	Must
US-05	Como admin , quiero crear productos en el catálogo con nombre, precio, código de barras y categoría.	5	Must
US-06	Como cajero , quiero escanear el código de barras de un producto para agregarlo al carrito.	5	Must
US-07	Como cajero , quiero buscar un producto por nombre para agregarlo al carrito cuando no tengo lector.	3	Must
US-08	Como cajero , quiero ver el subtotal, impuesto y total del carrito actualizado en tiempo real mientras agrego productos.	3	Must

ID	Historia de Usuario	Puntos	Prioridad
US-09	Como cajero , quiero cobrar en efectivo ingresando el monto recibido para que el sistema calcule el vuelto.	5	Must
US-10	Como cajero , quiero que al completar una venta se genere un comprobante imprimible con los ítems, total e impuesto.	5	Must
US-11	Como cajero , quiero cerrar el turno declarando el efectivo en caja para que el sistema calcule el cuadre.	3	Must

Total de puntos comprometidos para Sprint 1: **45 puntos**.

9. Alcance Preliminar: Plan de Sprints (MVP en 3 Iteraciones)

La restricción de **30 horas semanales totales** (6 personas $\times$ 5 h/semana) impone una disciplina estricta de priorización. El alcance del MVP se condensa en **3 Sprints de 4 semanas** cada uno, siguiendo el principio de entrega incremental de valor funcional verificable al final de cada iteración.

Visión de los Sprints

Sprint	Nombre	Entregable Principal	Capacidad
1	Infraestructura Base e Identidad	Kong funcionando + JWT end-to-end	120 h (4 semanas)
2	Catálogo y Punto de Venta	POS desktop + balanza + Catalog API	120 h (4 semanas)
3	Inventario y Eventos Asíncronos	FEFO + Kafka eventos + Alertas	120 h (4 semanas)

Sprint 1 — Infraestructura Base, API Gateway e Identidad Multi-Tenant

Duración: 4 semanas **Capacidad:** 120 horas **Objetivo:** Tener el esqueleto completo del sistema funcionando con autenticación segura end-to-end.

Épicas del Sprint:

1. **E1 — Repositorio y Convenciones:** Monorepo en GitHub, `.editorconfig`, `.gitignore`, estructura de carpetas (`services/`, `frontend/`, `gateway/`, `docs/`), ADR inicial de arquitectura, pipelines base de GitHub Actions (build + format check).

2. **E2 — Infraestructura Docker Compose:** Definición del `docker-compose.yml` con todos los contenedores: `postgres-tenant`, `zookeeper`, `kafka`, `kong`, `pgadmin`, `prometheus`. Red interna `minimarket-net`, volúmenes persistentes y health-checks (`pg_isready`).
3. **E3 — Kong 3.6 DB-less:** Declaración completa del archivo `gateway/kong.yml` con servicios, rutas y plugins: `jwt` (rutas protegidas), `rate-limiting` (endpoint `/auth/login`: máx. 10 req/min), `cors` (origen Electron). Verificación de que ninguna ruta protegida es alcanzable sin token.
4. **E4 — Tenant & Identity Service (ASP.NET Core 8):**
 - Proyecto ASP.NET Core 8 con arquitectura limpia: Domain, Application, Infrastructure, API.
 - Modelos de dominio: Tenant, TenantConfig (Owned Entity EF Core), Sucursal, User, RefreshToken.
 - Migraciones EF Core con Fluent API; seed de datos para tenant y usuario SUPER_ADMIN de demo.
 - Endpoints: POST `/auth/login`, POST `/auth/register`, POST `/auth/refresh`, POST `/auth/logout`, GET `/tenants`, POST `/tenants`, GET `/sucursales`, POST `/sucursales`.
 - Seguridad: BCrypt (passlib), JWT con claims `tenantId`, `sucursalId`, `role`, `tenantConfig`; `TenantResolutionMiddleware`.
5. **E5 — Frontend Electron (Autenticación):** Scaffold con *electron-vite* + React 18 + TypeScript 5. Pantallas: `LoginPage` y `RegisterPage` (multi-step con 3 pasos). `authStore` con *electron-store*. Interceptor Axios para renovación silenciosa de tokens. Flujo completo validado con Playwright.
6. **E6 — Testing Sprint 1:** Suite xUnit para dominio (Value Objects, reglas de negocio) e integración (`WebApplicationFactory`, PostgreSQL de test). Cobertura $\geq 70\%$. Test Playwright E2E: login correcto, login fallido, logout.

Criterio de Aceptación del Sprint: Un usuario puede registrar un minimarket, iniciar sesión desde la app Electron, recibir un JWT válido, navegar al dashboard y cerrar sesión; todo el tráfico HTTP pasa por Kong y el token es validado antes de llegar al microservicio.

Definición de Done (DoD):

- ☐ Todos los endpoints del sprint pasan tests de integración.
- ☐ Cobertura de tests unitarios $\geq 70\%$ en lógica de negocio de MS-1.
- ☐ Docker Compose levanta el entorno completo con un solo comando.
- ☐ El flujo login $\rightarrow$ dashboard $\rightarrow$ logout funciona end-to-end en Electron.
- ☐ Ninguna ruta protegida es alcanzable sin pasar por Kong.
- ☐ Pipeline de GitHub Actions ejecuta build + tests exitosamente en cada push.

Sprint 2 — Catálogo de Productos y Punto de Venta Desktop

Duración: 4 semanas **Capacidad:** 120 horas **Objetivo:** Operación completa del POS: desde escanear un producto hasta imprimir el ticket y cerrar el turno, incluyendo soporte de productos por peso.

Épicas del Sprint:**1. E7 — Catalog & Pricing Service (ASP.NET Core 8):**

- Modelos: *Categoria* (árbol auto-referenciado), *UnidadMedida*, *Producto*, *ProductoPrecio* (precio por sucursal, con fecha de vigencia), *ImagenProducto*.
- Migraciones EF Core en `postgres-catalog`.
- Endpoints CRUD completos para categorías, unidades de medida y productos. Endpoint `GET /productos/barcode/{code}` para búsqueda instantánea por código de barras desde el POS.
- Endpoint de precios por sucursal con soporte de vigencia de precios.
- Publica evento Kafka `product.created` y `product.price_updated` al crear o actualizar productos.

2. E8 — POS & Cart Service (ASP.NET Core 8):

- Modelos: *TurnoCaja*, *Venta*, *VentaItem*, *Pago*, *Anulacion*.
- Flujo de turno: apertura (`POST /turnos`), cierre (`PATCH /turnos/{id}/cerrar`) con cuadre de caja.
- Flujo de venta: agregar ítem, quitar ítem, aplicar descuento, procesar cobro (efectivo, tarjeta, mixto), calcular vuelto.
- Al completar una venta: publica evento Kafka `sale.completed` con payload completo de ítems.
- Endpoint de anulación con validación de autorización por rol (`ADMINISTRADOR` requerido).

3. E9 — Frontend POS (Electron):

- **Pantalla POS:** panel de escaneo (input de código de barras con foco automático) + panel de carrito (lista de ítems, subtotales, total con impuesto).
- **Integración de balanza (hardware):** el proceso *main* de Electron lee el peso desde el puerto serie (USB HID) mediante el paquete `serialport`. El peso se expone al renderer vía `contextBridge` IPC como evento `scale:weight`.
- **Modal de cobro:** tres pestañas (Efectivo / Tarjeta / Mixto), cálculo de vuelto en tiempo real.
- **Impresión de ticket:** generación en HTML + CSS e impresión silenciosa vía `webContents.print()`.
- **Apertura y cierre de turno:** pantalla dedicada con monto inicial y reporte de cierre.

4. E10 — Testing Sprint 2: Tests xUnit para servicios de dominio del catálogo y del POS. Tests de integración para los endpoints de venta. Tests Playwright E2E: flujo de venta completo (escanear, cobrar, imprimir).

Criterio de Aceptación del Sprint: Un cajero puede abrir su turno, escanear productos (incluyendo uno de peso variable registrando el peso desde la balanza), procesar el cobro en efectivo, imprimir el ticket y cerrar el turno; la venta queda registrada en la base de datos y el evento `sale.completed` es publicado en Kafka.

Sprint 3 — Inventario, Regla FEFO y Arquitectura de Eventos Kafka

Duración: 4 semanas **Capacidad:** 120 horas **Objetivo:** Completar el ciclo de vida del inventario con trazabilidad por lote, consumir eventos de Kafka para descontar stock automáticamente y generar alertas proactivas al equipo de tienda.

Épicas del Sprint:**1. E11 — Warehouse & Inventory Service (ASP.NET Core 8):**

- Modelos: Refrigerador (con capacidad en m^3 y m^2), Lote (número de lote, fecha de vencimiento, refrigerador asignado), ProductoStock, MovimientoInventario (tipos: ENTRADA, SALIDA, TRANSFERENCIA, MERMA), Merma.
- Regla **FEFO**: al descontar stock, el sistema selecciona el lote con `fecha_vencimiento` más próxima. Implementado como método de dominio en el servicio de aplicación.
- Endpoints: recepción de mercadería, ajuste manual de stock, registro de mermas, consulta de lotes próximos a vencer.
- **Kafka Consumer** (grupo de consumo `warehouse-service`): consume evento `sale.completed` → ejecuta descuento FEFO → si `stock_actual < stock_minimo`, publica `stock.low`.
- **IHostedService** de fondo: job diario que revisa lotes con `fecha_vencimiento ≤ NOW() + 3 días` y publica `product.expiring_soon` en Kafka.

2. E12 — Analytics & Notification Service (ASP.NET Core 8):

- Kafka Consumers para `sale.completed`, `stock.low` y `product.expiring_soon`.
- Al consumir `sale.completed`: actualiza métricas de ventas del día y recalcula promedio móvil de 7 días.
- Al consumir `stock.low` o `product.expiring_soon`: genera registro en `AlertasGeneradas` con severidad (`WARNING` / `CRITICAL`) y crea notificación para el Administrador de la sucursal.
- API de alertas activas: `GET /alerts?tenantId=&resolved=false`.

3. E13 — Frontend Dashboard de Inventario y Alertas (Electron):

- Pantalla de inventario: tabla de productos con stock actual, stock mínimo y badge de alerta si `stock_actual < stock_minimo`.
- Pantalla de lotes: lista de lotes con fecha de vencimiento, resaltando en rojo los que vencen en menos de 3 días.
- Notificaciones en la barra lateral: badge con contador de alertas no resueltas; panel lateral de alertas al hacer clic.
- Pantalla de registro de merma con formulario de selección de lote y cantidad.

4. E14 — Observabilidad y Cierre del MVP: Integración de *prometheus-dotnet* en todos los servicios ASP.NET Core para exponer métricas en `/metrics`. Configuración de *Prometheus* y dashboard básico en *Grafana* con alertas de latencia y tasa de errores HTTP.**5. E15 — Testing Sprint 3 y Cobertura Final:** Tests xUnit para la lógica FEFO (casos borde: sin lotes disponibles, lotes mixtos, lote único). Tests de integración para los consumers de Kafka (usando *Testcontainers* o mocks de `IConsumer<>`). Suite E2E Playwright: flujo completo de alerta (venta → Kafka → inventario bajo

→ notificación visible en Electron).

Criterio de Aceptación del Sprint (MVP Completo): Al completar una venta en el POS, el stock se descuenta automáticamente respetando FEFO; si el stock cae por debajo del mínimo, el Administrador ve una alerta en el dashboard de Electron en menos de 5 segundos. Los lotes próximos a vencer aparecen marcados. El sistema completo (7 contenedores) arranca con `docker compose up` en menos de 2 minutos.

10. Roadmap — Sprints Futuros

Más allá del MVP cubierto por los 3 sprints anteriores, el siguiente roadmap proyecta la evolución del producto hacia las épicas Should y Could del backlog.

Sprint	Objetivo	Épicas
Sprint 4	Cobro con tarjeta, integración con Terminal POS y balanza física	EP-02, EP-05
Sprint 5	Cumplimiento fiscal completo (DTE, folios, integración Entidad Fiscal)	EP-05
Sprint 6	Dashboards, reportes, exportación PDF/Excel	EP-08
Sprint 7	Órdenes de compra, Landed Cost, gestión de proveedores	EP-07
Sprint 8	Programa de lealtad, cliente afiliado, cupones	EP-09
Sprint 9	Notificaciones, FX en tiempo real, tracking de importaciones	EP-10, EP-11

11. Restricciones y Recursos

11.1. Equipo de Desarrollo

El proyecto es ejecutado por un equipo de **6 integrantes Full-Stack**, principio que implica que todos los miembros participan en todas las capas del sistema (infraestructura, backend, frontend, pruebas). No existen roles exclusivos ni silos de conocimiento.

Integrante	Área de Énfasis Principal	Responsabilidades adicionales
Diego	DevOps / Arquitectura / Kong	Docker Compose, CI/CD, C4 Diagrams
Sebastián	Backend Lead / DDD / Seguridad	JWT, EF Core Fluent API, Domain Design
Martín	Base de Datos / EF Core	Migraciones, ERDs, Seeds, Indices PG
Rodrigo	Frontend Lead / UX / Electron	React, IPC, Hardware (balanza)
Daniel	QA Lead / xUnit / Playwright	Cobertura, E2E, Estrategia de Testing
Nicolás	Backend / Kafka / CI-CD	Consumers, Producers, Pipeline Actions

11.2. Restricción de Capacidad

Restricción crítica de tiempo: Cada integrante dispone de **5 horas semanales** dedicadas al proyecto. Esto genera una capacidad total de **30 horas semanales** y **120 horas por Sprint** (4 semanas × 30 h/semana). Las tareas son diseñadas para no exceder bloques de **3–5 horas** por persona por semana, incluidas reuniones Scrum.

Concepto	Semana	Sprint (4 semanas)
Personas en el equipo	6	6
Horas por persona	5 h	20 h
Total equipo	30 h	120 h
Deducción Scrum (Daily + Retro + Planning)	–2 h/persona	–8 h/persona
Horas productivas netas	18 h	72 h

11.3. Stack Tecnológico Oficial

Capa	Tecnología	Versión / Notas
Frontend Desktop	Electron + React	React 18+, TypeScript 5+, electron-vite
Estado cliente	electron-store	Persistencia local de tokens y config
Backend	ASP.NET Core 8	Minimal APIs + Controllers, C# 12
ORM / Migraciones	Entity Framework Core 8	Fluent API, Code-First, AsyncEnumerable
Base de Datos	PostgreSQL 16	Patrón <i>Database-per-Service</i> (7 BD)
API Gateway	Kong 3.6	Modo DB-less, declarativo (<code>kong.yml</code>)
Bus de Eventos	Apache Kafka	+ Zookeeper; producer/consumer en .NET
Testing Backend	xUnit 2+	FluentAssertions + Moq + Coverlet
Testing E2E	Playwright	<code>_electron.launch()</code> , CI-ready
Contenedores	Docker Compose	Entorno completo en un único comando
CI/CD	GitHub Actions	Build → Test → Coverage → Docker Push

11.4. Recursos de Hardware

- **Balanza de pesaje:** conexión USB HID o serial RS-232, leída desde el proceso *main* de Electron mediante la librería `serialport`.
- **Impresora de tickets:** soporte de impresión silenciosa vía `webContents.print()` de Electron con plantilla HTML/CSS.
- **Estaciones de desarrollo:** cada integrante trabaja en su equipo local; el entorno

completo se levanta con `docker compose up -build`.

12. Alineación con el Estándar SWEBOK

El estándar **SWEBOK** (**Software Engineering Body of Knowledge**) de la IEEE guía las buenas prácticas en la ingeniería de software. La documentación de GlobalMart OS se ha estructurado siguiendo las siguientes áreas de conocimiento:

- **Requisitos de Software (REQ):** Las secciones de *Análisis de Requerimientos* (funcionales y no funcionales), *Reglas de Negocio* y *Casos de Uso* cubren los procesos de elicitación, análisis y especificación de requisitos.
- **Diseño de Software (DES):** La arquitectura de microservicios, el modelo multi-tenant y los patrones DDD (*Domain-Driven Design*) documentados en el *Alcance Preliminar* y en los diagramas C4 planificados para el repositorio, cubren el área de diseño.
- **Pruebas de Software (TST):** La estrategia de testing definida en cada Sprint (xUnit, Playwright, cobertura $\geq 70\%$) y los criterios de aceptación detallados en el *Backlog* abordan esta área de conocimiento.
- **Gestión de Proyectos de Software (MGT):** La planificación con Scrum, la estimación de capacidad (30 h/semana), los Sprints de 4 semanas y el *Backlog priorizado* reflejan las prácticas de gestión de proyectos.
- **Procesos de Software (PRO):** La metodología Scrum, las revisiones de código (PRs) y el pipeline de CI/CD con GitHub Actions documentan los procesos de desarrollo.

Esta estructura asegura que la documentación cumpla con los estándares internacionales de la IEEE, garantizando la calidad, trazabilidad y mantenibilidad del proyecto.

13. Estructura del Repositorio para Documentación

Para garantizar la centralización, trazabilidad y conservación de toda la documentación generada a lo largo del proyecto, se ha definido la siguiente estructura de carpetas en el repositorio de GitHub:

```
/docs
01-perfil-proyecto/
  GlobalMart_ProjectDoc.pdf      (Documento actual, entregable Sprint 0)
02-analisis-sprint0/
  reglas_negocio.md
  requerimientos.md
  casos_uso.md
  matriz_trazabilidad.md
```

```
03-requisitos/
  requisitos_funcionales.md
  requisitos_no_funcionales.md
04-arquitectura/
  diagrama_c4_contexto.puml
  diagrama_c4_contenedores.puml
  diagrama_c4_componentes.puml
  adrs/                                (Architecture Decision Records)
    adr-001-microservicios.md
    adr-002-multi-tenant.md
05-planos-sprints/
  sprint-01-backlog.md
  sprint-02-backlog.md
  sprint-03-backlog.md
06-minutas/
  daily-01-10-2026.md
  sprint-review-01.md
  sprint-retro-01.md
07-manuales/
  manual_usuario.md
  manual_tecnico.md
README.md                             (Índice y propósito de cada carpeta)
```

Esta estructura permite:

- **Trazabilidad:** Cada documento está vinculado a su Sprint y área de conocimiento.
- **Conservación:** Todos los artefactos se mantienen versionados en el repositorio.
- **Accesibilidad:** Cualquier integrante puede localizar la documentación necesaria.

14. Riesgos Identificados

R1 — Curva de aprendizaje del ecosistema (30 h/semana)

Clasificación: Crítico. **Impacto:** Alto.

El stack seleccionado representa una curva significativa con tiempo limitado.

Mitigación: Bootcamp interno en Semana 1, templates de microservicios estandarizados y PRs obligatorios.

R2 — Comunicación asíncrona con Kafka

Clasificación: Alto. **Impacto:** Alto.

Riesgo de procesamiento duplicado de eventos (doble descuento de stock).

Mitigación: Consumers idempotentes y Dead-Letter Queues (DLQ) para reintentos.

R3 — Integración de hardware en Electron

Clasificación: Moderado. **Impacto:** Medio.

Lectura de peso desde balanza USB/Serial depende de drivers volátiles.

Mitigación: Interfaz abstracta con Mock Adapter en dev y fallback manual en POS.

R4 — Fragmentación del conocimiento en equipo Full-Stack

Clasificación: Moderado. **Impacto:** Medio.

El modelo Full-Stack con horas limitadas puede generar pérdida de contexto.

Mitigación: Matriz de responsabilidad, Daily Scrum, ADRs y Sprint Reviews.

R5 — Deuda técnica por presión de tiempo en lógica FEFO

Clasificación: Bajo (vigilado). **Impacto:** Alto.

Errores en FEFO pueden despachar productos vencidos.

Mitigación: Método de dominio puro, tests unitarios de casos borde y revisión doble (PR).